

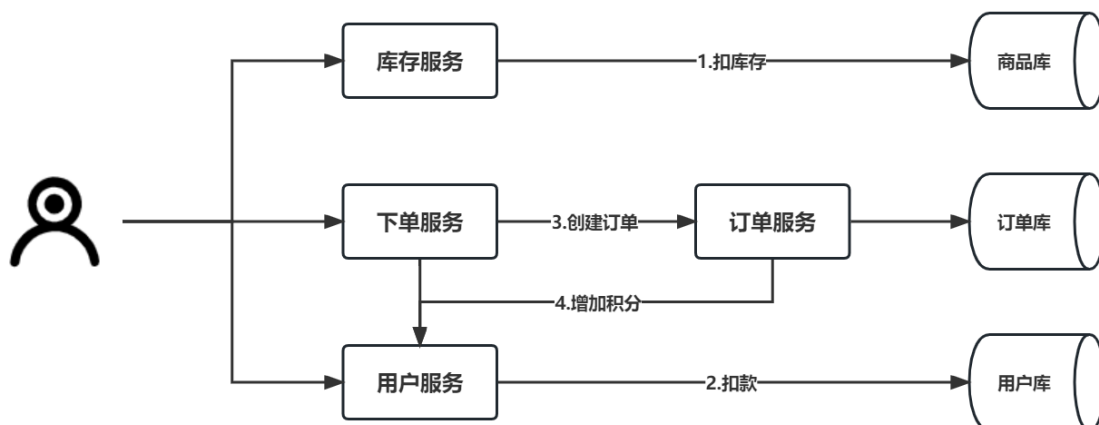
微服务SpringCloudAlibaba

授课目标

- 目标01-理解分布式事务产生场景
- 目标02-掌握分布式事务解决方案基本原理
- 目标03-理解Seata事务模式基本原理
- 目标04-掌握Seata分布式事务的基本使用

第7章 Seata分布式事务

在微服务架构中分布式事务是我们绕不开问题，比如电商项目下单业务中，先要调用库存微服务扣减库存，然后调用订单微服务下单，在订单微服务中又通过Feign调用用户微服务，增加用户积分。最终才能完成一个下单业务，而在不同微服务中可能又要操作不同的数据库，**如何保证这种业务的事务一致性？**



7.1 分布式事务简介

7.1.1 事务简介

1) 什么是事务？

事务可以看做是一次大的活动，它由不同的小活动组成，这些活动要么全部成功，要么全部失败。

2) 本地事务/本地事务

在单体架构是通过**关系型数据库**来控制事务，这是利用数据库本身的事务特性来实现的，因此叫数据库事务，由于应用主要靠关系数据库来控制事务，而数据库通常和应用在同一个服务器，所以基于关系型数据库的事务又被称为本地事务。

数据库事务在实现时会将一次事务的所有操作全部纳入到一个不可分割的执行单元，该执行单元的所有操作要么都成功，要么都失败，只要其中任一操作执行失败，都将导致整个事务回滚。

事务四大特性ACID：

- **原子性 (Atomicity)**：原子性是指事务是一个不可分割的工作单位，事务中的操作要么都发生，要么都不发生。
- **一致性 (Consistency)**：事务前后数据的完整性必须保持一致
- **隔离性 (Isolation)**：多个用户并发访问数据库时，一个用户的事务不能被其它用户的事务所干扰，多个并发事务之间数据要相互隔离。**隔离性由隔离级别保障！**
- **持久性 (Durability)**：一个事务一旦被提交，它对数据库中数据的改变就是永久性的，接下来即使数据库发生故障也不应该对其有任何影响。

7.1.2 什么是分布式事务？

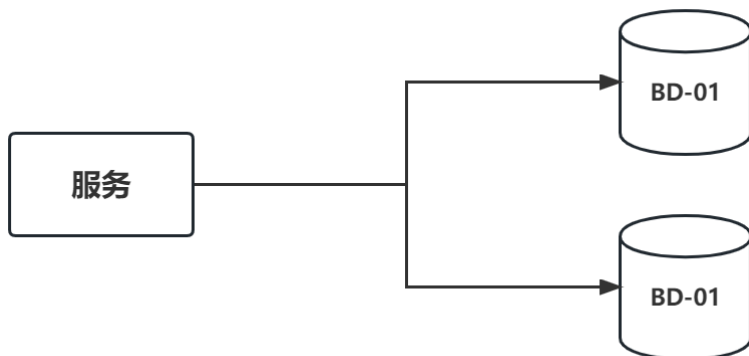
分布式事务是指在微服务架构下保证事务的特性，包括原子性、一致性、隔离性和持久性。

通俗地说就是：一个**操作单元**，由若干分支操作组成，这些**分支操作分属不同应用**，分布在不同服务器上。分布式事务需要保证这些分支操作要么全部成功，要么全部失败。分布式事务与普通事务一样，就是为了保证对数据库数据操作的一致性。

7.1.3 分布式事务产生场景

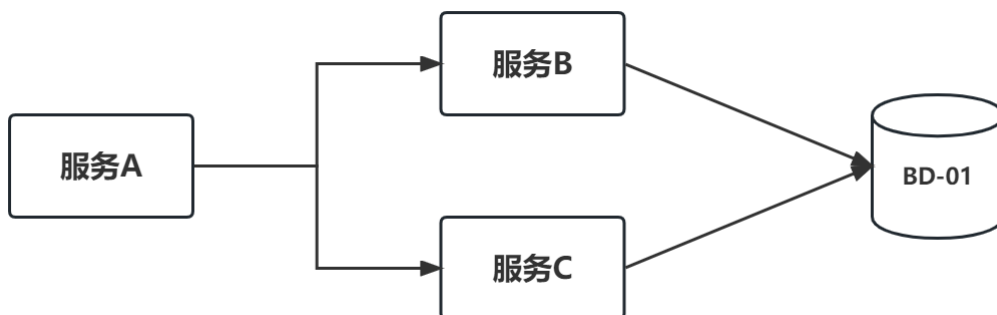
1) 单一服务分布式事务

单一服务的分布式事务，是指单体架构下，一个服务操作，不涉及服务间的相互调用，但是这个服务会涉及多个数据库资源的访问。



2) 多服务分布式事务

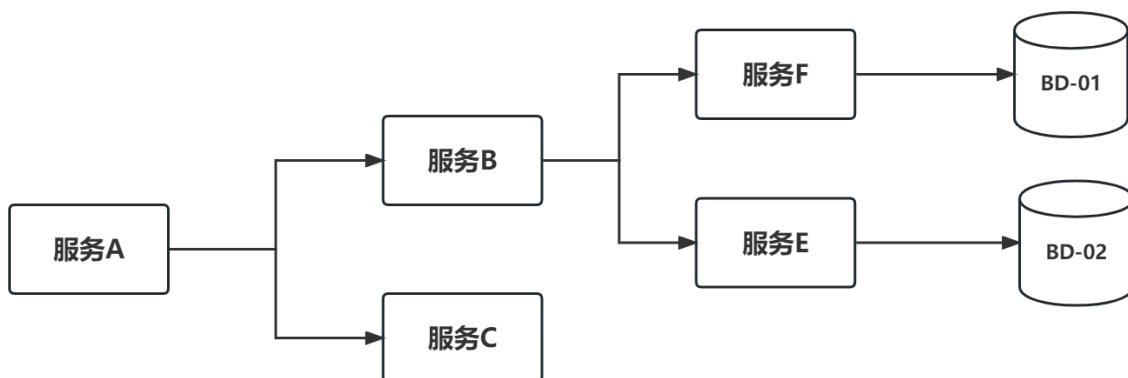
如果一个服务业务逻辑，需要调用另外一个服务，比如下单服务，需要调用库存服务扣减库存，这时事务就需要跨多个服务了，在这种情况下，起始于某个服务的事务在调用另一个服务的时候需要以一定的机制流转 to 另外一个服务，从而是被调用的服务访问的数据库资源也能纳入到该事务的管理中。



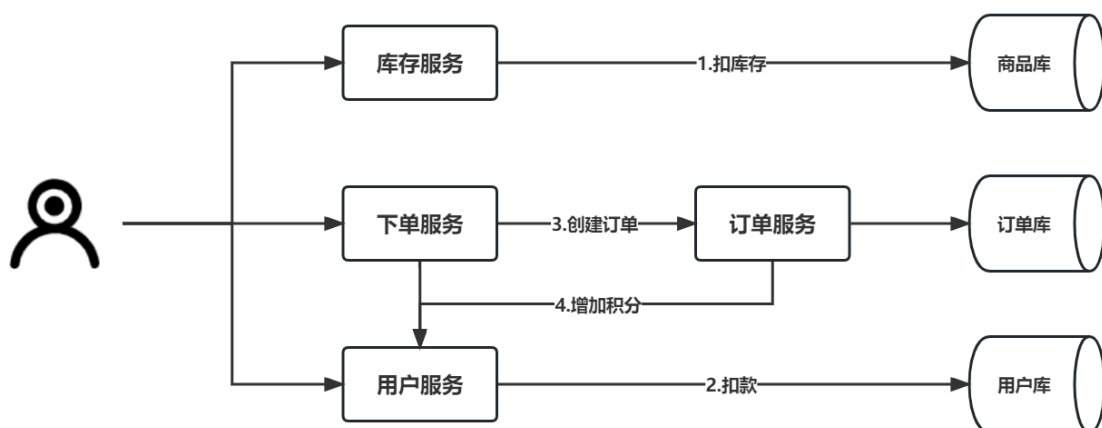
3) 多服务多数据源分布式事务

如果将上面两种场景结合，服务可以调用多个数据库资源，而一个服务又可以调用其他服务，完成一个业务操作。

多个微服务操作不同的数据库，整个分布式事务的参与者将会组成一个树形的拓扑结构，在一个跨服务的分布式事务中，事务的发起者和提交者均系同一个，他可以是整个调用的客户端，也可以是客户端最先调用的那个服务。



举个栗子：下单业务需要调用库存微服务扣减库存，库存微服务操作商品库，下单业务有需要调用用户微服务，实现增加用户积分操作，用户微服务又调用用户库，这样一个下单业务，就需要调用多个微服务，访问多个数据库。



7.2 分布式事务解决方案

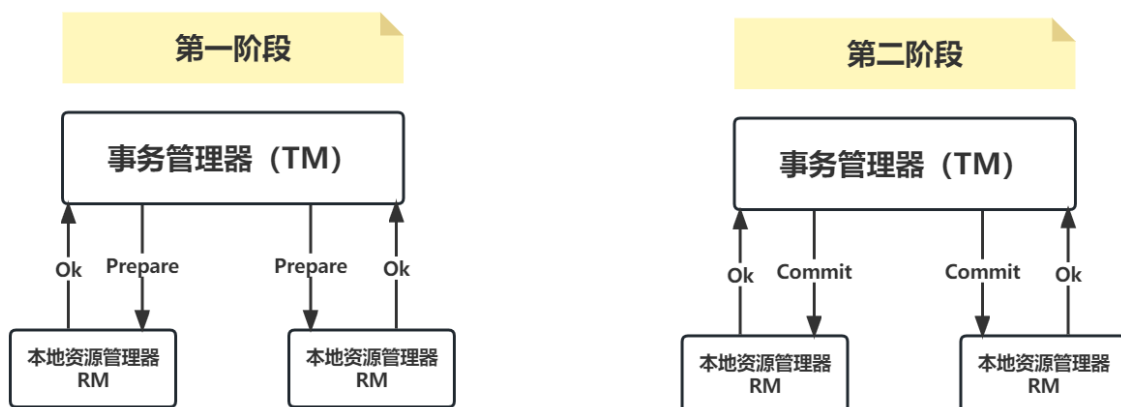
在分布式系统中，要实现分布式事务，也就如下几种解决方案，2PC、TCC、本地消息表、事务消息。

7.2.1 XA协议

XA 协议是由 X/Open 组织提出的分布式事务处理规范，主要定义了事务管理器 TM 和局部资源管理器 RM 之间的接口。目前主流的数据库，比如 Oracle、DB2 都是支持 XA 协议的。MySQL 从 5.0 版本开始，InnoDB 存储引擎已经支持 XA 协议。

XA中大致分为两部分：**事务管理器**和**本地资源管理器**。其中本地资源管理器往往由数据库实现，而事务管理器作为全局的调度者，负责各个本地资源的提交和回滚。

XA实现分布式事务的原理如下：



XA协议是一个典型的2PC，下面我们介绍一下2PC。

7.2.2 两阶段提交(2PC)

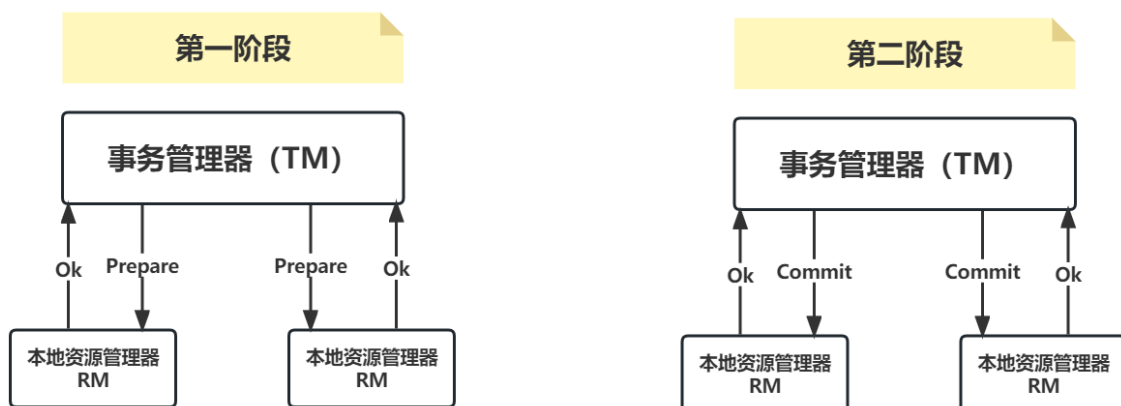
2PC 即两阶段提交协议，是将整个事务流程分为两个阶段，准备阶段（Prepare phase）、提交阶段（commit phase）。部分关系数据库如 Oracle、MySQL都支持两阶段提交协议。

在计算机中部分关系数据库如 Oracle、MySQL 支持两阶段提交协议，如下图：

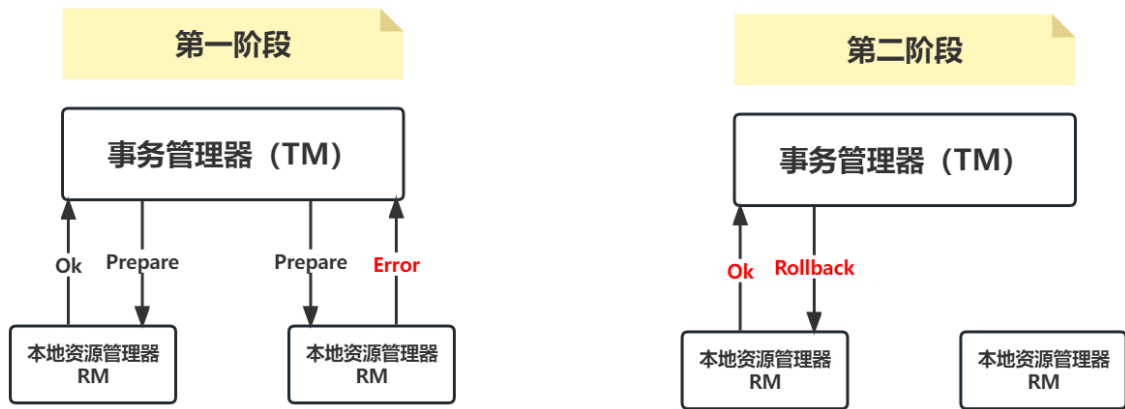
1. 准备阶段（Prepare phase）：事务管理器给每个参与者发送 Prepare 消息，每个数据库参与者在本地执行事务，并写本地的 Undo/Redo 日志，此时事务没有提交。（Undo 日志是记录修改前的数据，用于数据库回滚，Redo 日志是记录修改后的数据，用于提交事务后写入数据文件）
2. 提交阶段（commit phase）：如果事务管理器收到了参与者的执行失败或者超时消息时，直接给每个参与者发送回滚（Rollback）消息；否则，发送提交（Commit）消息；参与者根据事务管理器的指令执行提交或者回滚操作，并释放事务处理过程中使用的锁资源。注意：**必须在最后阶段释放锁资源。**

下图展示了2PC的两个阶段，分成功和失败两个情况说明：

成功情况



失败情况



举个栗子：

- 雄哥和老王好久不见，老友约起聚餐，饭店老板要求先买单，才能出票。
- 这时雄哥和老王分别抱怨近况不如意，囊中羞涩，都不愿意请客，这时只能AA。只有雄哥和老王都付款，老板才能出票安排就餐。但由于雄哥和老王都是铁公鸡，形成了尴尬的一幕：
 1. 准备阶段：老板要求雄哥付款，雄哥付款。老板要求老王付款，老王付款。
 2. 提交阶段：老板出票，两人拿票纷纷落座就餐。

例子中，形成吃饭聚餐就是一个事务，若雄哥或老王其中一人拒绝付款，或钱不够，店老板都不会给出票，并且会把已收款退回。

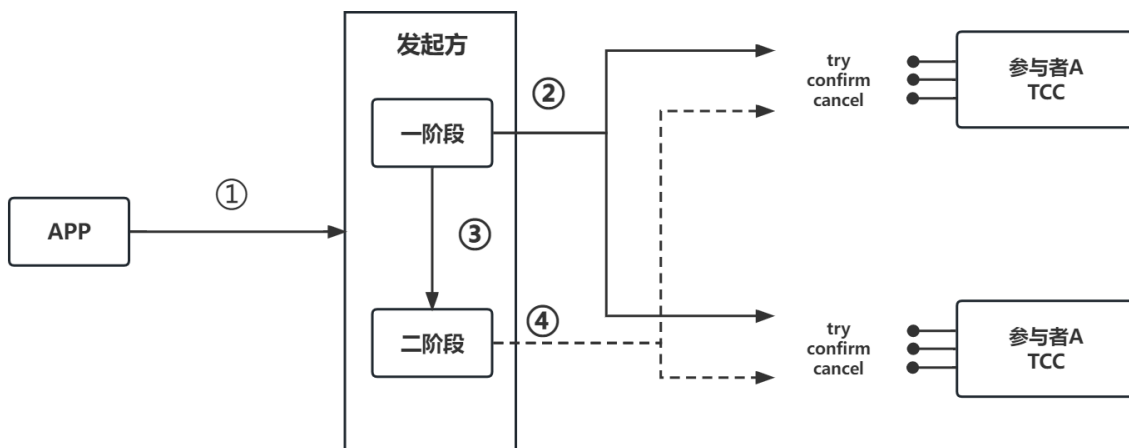
整个事务过程由**事务管理器**和**参与者**组成，**店老板就是事务管理器**，**雄哥、老王就是事务参与者**，事务管理器负责决策整个分布式事务的提交和回滚，事务参与者负责自己本地事务的提交和回滚。

2PC方案缺陷：

1. **性能问题**：所有参与者在事务提交阶段处于同步阻塞状态，占用系统资源，容易导致性能瓶颈。
2. **可靠性问题**：如果协调者存在**单点故障**问题，如果协调者出现故障，参与者将一直处于锁定状态。
3. **数据一致性问题**：在阶段 2 中，如果发生局部网络问题，一部分事务参与者收到了提交消息，另一部分事务参与者没收到提交消息，那么就导致了节点之间数据的不一致。

7.2.3 补偿事务 (TCC)

TCC 就是采用的补偿机制，其核心思想是：针对每个操作，都要编写一个与其对应的确认和补偿（撤销）操作逻辑。



TCC分为三个阶段。

- Try 阶段主要是对业务系统做检测及资源预留。
- Confirm 阶段主要是对业务系统做确认提交，Try阶段执行成功并开始执行 Confirm阶段时，默认 Confirm阶段是不会出错的。
 - 只要Try成功，Confirm一定成功
- Cancel 阶段主要是在业务执行错误，需要回滚的状态下执行的业务取消，预留资源释放。

举个栗子：假设雄哥要向老王转账，在转账方法，里面依次调用

- 第一步：首先在 Try 阶段，要先调用远程接口把 雄哥和老王的钱给冻结起来。
- 第二步：在 Confirm 阶段，执行远程调用的转账的操作，转账成功进行解冻。
- 第三步：如果第2步执行成功，那么转账成功，如果第二步执行失败，则调用远程冻结接口对应的解冻方法 (Cancel)。

TCC优点：

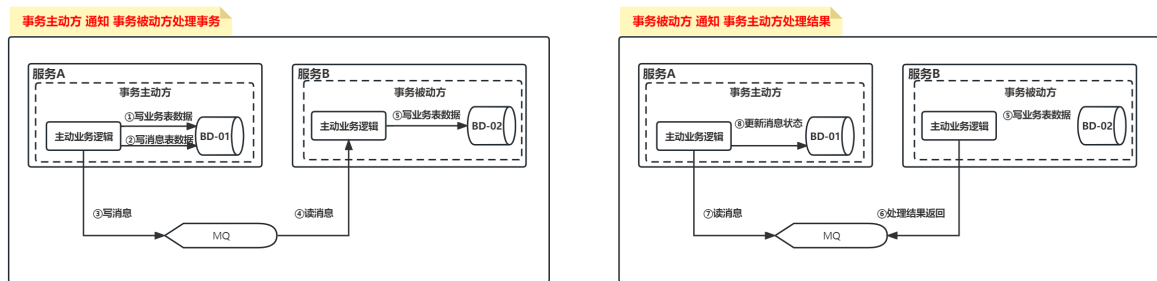
- **性能提升：**具体业务来实现控制资源锁的粒度变小，不会锁定整个资源。
- **流程简单：**跟2PC比起来，实现以及流程相对简单了一些。
- **可靠性高：**解决了 XA 协议的协调者单点故障问题，由主业务方发起并控制整个业务活动，业务活动管理器也变成多点。

TCC缺点：

- **代码编写的复杂度较高：**TCC属于应用层的一种补偿方式，所以需要程序员在实现的时候多写很多补偿的代码
- **使用场景局限：**在一些场景中，一些业务流程可能用TCC不太好定义及处理

7.2.4 本地消息表

本地消息表的方案最初是由 eBay 提出，核心思路是**将分布式事务拆分成本地事务进行处理**。



消息生产方，需要额外建一个消息表，并记录消息发送状态。消息表和业务数据要在一个事务里提交，也就是说他们要在一个数据库里面。然后消息会经过MQ发送到消息的消费方。如果消息发送失败，会进行重试发送。

消息消费方，需要处理这个消息，并完成自己的业务逻辑。此时如果本地事务处理成功，表明已经处理成功了，如果处理失败，那么就会重试执行。如果是业务方法多次执行失败，可以给生产方发送一个业务补偿消息，通知生产方进行回滚等操作。

生产方和消费方定时扫描本地消息表，把还没处理完成的消息或者失败的消息再发送一遍。

这种方案遵循BASE理论，采用的是最终一致性，即不会出现像2PC那样复杂的实现（当调用链很长的時候，2PC的可用性是很低），也不会像TCC那样可能出现确认或者回滚不了的情况。

本地消息表事务优缺点：

- 优点：避免了分布式事务，实现了最终一致性
- 缺点：
 - 与具体的业务场景绑定，耦合性强
 - 消息数据与业务数据同库，占用业务系统资源。
 - 业务系统在使用关系型数据库的情况下，消息服务性能会受到关系型数据库并发性能的局限

7.2.5 Rocket MQ事务消息

RocketMQ支持事务消息，事务消息的方式类似于采用的二阶段提交。

业务方法内向MQ提交两次请求，一次**发送消息**和一次**确认消息**。如果确认消息发送失败了RocketMQ会定期扫描消息集群中的事务消息，这时候发现了Prepared消息，它会向消息发送者确认，所以生产方需要实现一个check回查接口，RocketMQ会根据发送端设置的策略来决定是回滚还是继续发送确认消息。这样就保证了消息发送与本地事务同时成功或同时失败。

RocketMQ的事务消息分为两个流程：

1. 正常事务消息的发送及提交
2. 二事务消息的补偿流程

1) 正常事务消息的发送提交

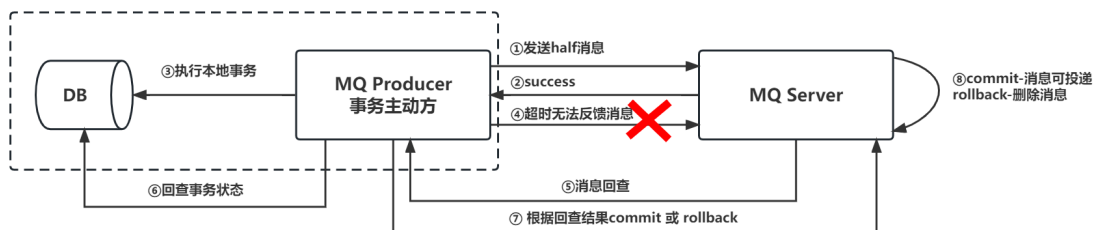


事务消息正常情况下发送放向MQ Server进行二次确认，步骤如下：

- 步骤1-发送方向 MQ 服务端(MQ Server)发送 half 消息
- 步骤2-MQ Server 将消息持久化成功之后，向发送方 ack 确认消息已经发送成功
- 步骤3-发送方开始执行本地事务逻辑
- 步骤4-发送方根据本地事务执行结果向 MQ Server 提交二次确认（commit 或是 rollback）

- 步骤5-MQ Server 收到 commit 状态则将半消息标记为可投递，订阅方最终将收到该消息；
 - MQ Server 收到 rollback 状态则删除半消息，订阅方将不会接受该消息

2) 事务消息补偿流程



正常情况下，事务主动消息恢复

步骤4提交的二次确认超时未能到达 MQ Server，此时MQ Server发起消息回查，此时处理步骤如下：

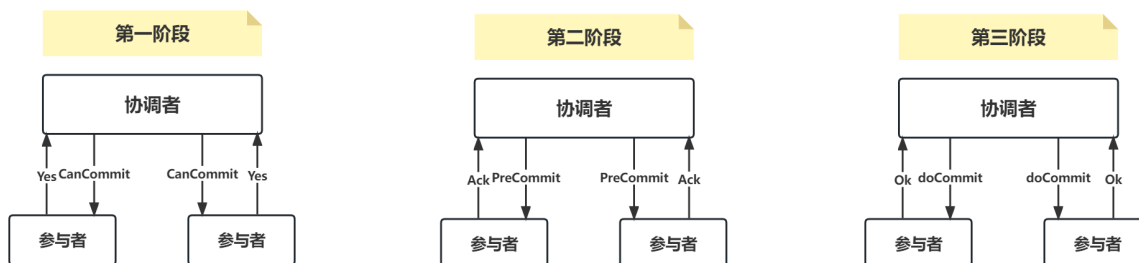
- 步骤5-MQ Server 对该消息发起消息回查
- 步骤6-发送方收到消息回查后，需要检查对应消息的本地事务执行的最终结果
- 步骤7-发送方根据检查得到的本地事务的最终状态再次提交二次确认
- 步骤8-MQ Server基于 commit/rollback 对消息进行投递或者删除

MQ事务消息优缺点：

- 优点，实现了最终一致性，不需要依赖本地数据库事务
- 缺点，
 - 实现难度大
 - 主流MQ不支持（RabbitMQ、Kafka不支持事务消息）

7.2.6 三阶段提交（3PC）

三阶段提交（Three-phase commit）也叫三阶段提交协议（Three-phase commit protocol）是二阶段提交（2PC）的改进版本。



3PC相比于2PC有两个改动点：

- 第一：引入超时机制。同时在协调者和参与者中都引入超时机制。
- 第二：在第一阶段和第二阶段中插入一个准备阶段。保证了在最后提交阶段之前各参与节点的状态是一致的。

除了引入超时机制之外，3PC把2PC的准备阶段再次一分为二，这样三阶段提交就有 CanCommit、PreCommit、DoCommit 三个阶段。

CanCommit阶段：3PC的CanCommit阶段其实和2PC的准备阶段很像。协调者向参与者发送commit请求，参与者如果可以提交就返回Yes响应，否则返回No响应。

PreCommit阶段：协调者根据参与者的反应情况来决定是否可以进行事务的PreCommit操作。根据响应情况，有以下两种可能。

- 假如协调者从所有的参与者获得的反馈都是Yes响应，那么就会执行事务的预执行
- 假如有任何一个参与者向协调者发送了No响应，或者等待超时之后，协调者都没有接到参与者的响应，那么就执行事务的中断

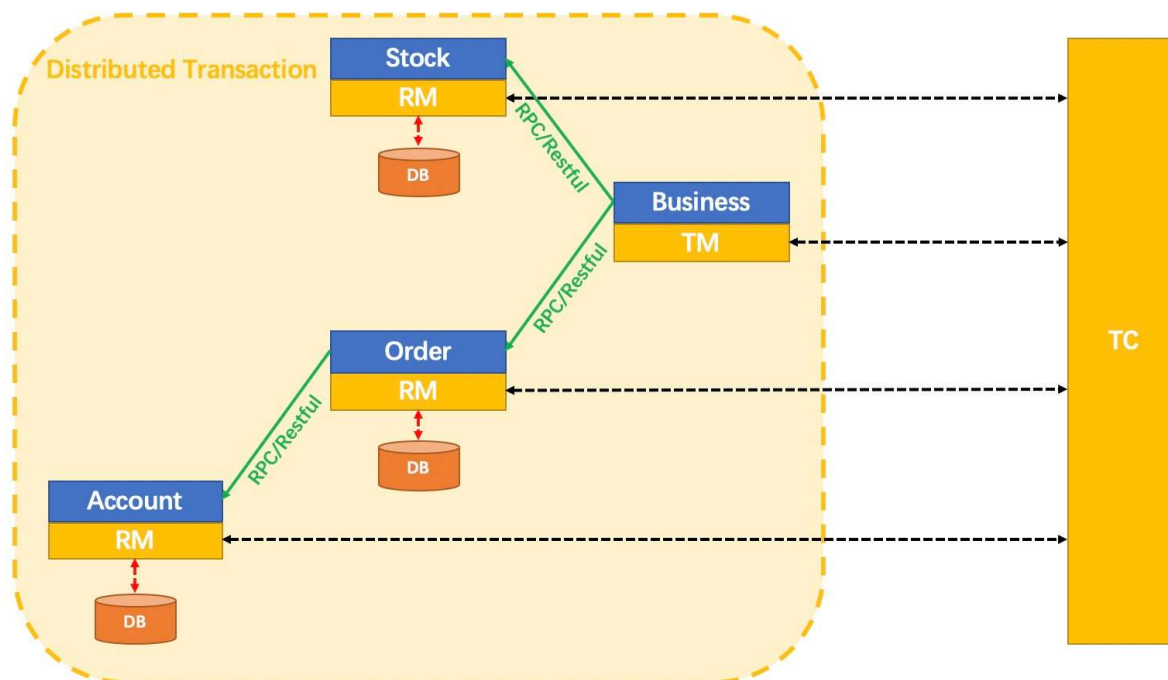
doCommit阶段：该阶段进行真正的事务提交，也可以分为以下两种情况。

- 执行提交：协调者接收到所有参与者的ack响应之后，完成事务
- 中断事务：协调者没有接收到参与者发送的ACK响应，可能是接受者发送的不是ACK响应，也可能响应超时，那么就会执行中断事务

7.2 Seata简介

Seata官网：<http://seata.io/zh-cn/>

Seata 是一款开源的分布式事务解决方案，致力于在微服务架构下提供高性能和简单易用的分布式事务服务。Seata 将为用户提供了 AT、TCC、SAGA 和 XA 事务模式，为用户打造一站式的分布式解决方案。



7.2.1 Seata术语

- Transaction Coordinator (TC)：事务协调器，它是独立的中间件，需要独立部署运行，它维护全局事务的运行状态，接收 TM 指令发起全局事务的提交与回滚，负责与 RM 通信协调各分支事务的提交或回滚。
- Transaction Manager (TM)：事务管理器，TM 需要嵌入应用程序中工作，它负责开启一个全局事务，并最终向 TC 发起全局提交或全局回滚的指令。
- Resource Manager (RM)：控制分支事务，负责分支注册、状态汇报，并接收事务协调器 TC 的指令，驱动分支（本地）事务的提交和回滚。

7.2.2 Seata分布式事务模式

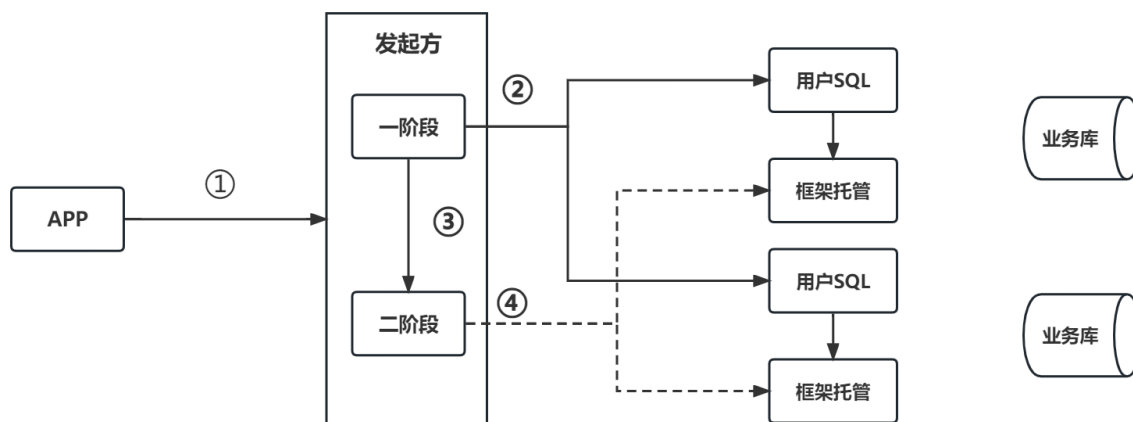
Seata提供了AT、TCC、SAGA与XA事务模式。

7.2.2 AT模式

AT模式是Seata的一大特色，AT对业务代码完全无侵入性，使用非常简单，改造成本低。我们只需要关注自己的业务SQL，Seata会通过分析我们业务SQL，反向生成回滚数据，在AT模式下，用户只需关心自己的“业务SQL”。

AT模式分为两个阶段：

- 一阶段，所有参与事务的分支，本地事务Commit 业务数据 和 写入回滚日志（undoLog）
- 二阶段，**事务协调者**根据所有分支的情况，决定本次全局事务是Commit 还是 Rollback



第一阶段：Seata 的 JDBC 数据源代理通过对业务 SQL 的解析，把业务数据在更新前后的数据镜像组织成回滚日志，利用本地事务的 ACID 特性，将业务数据的更新和回滚日志的写入在同一个本地事务中提交。

- 这样可以保证：任何提交的业务数据的更新一定有相应的回滚日志存在。
- 基于这样的机制，分支的本地事务便可以在全局事务的第一阶段提交，并马上释放本地事务锁定的资源。
- 这也是Seata和XA事务的不同之处，两阶段提交往往对资源的锁定需要持续到第二阶段实际的提交或者回滚操作，而有了回滚日志之后，可以在第一阶段释放对资源的锁定，降低了锁范围，提高效率，即使第二阶段发生异常需要回滚，只需找对undolog中对应数据并反解析成SQL来达到回滚目的。
- 同时Seata通过代理数据源将业务SQL的执行解析成undolog来与业务数据的更新同时入库，达到了对业务无侵入的效果。

第二阶段

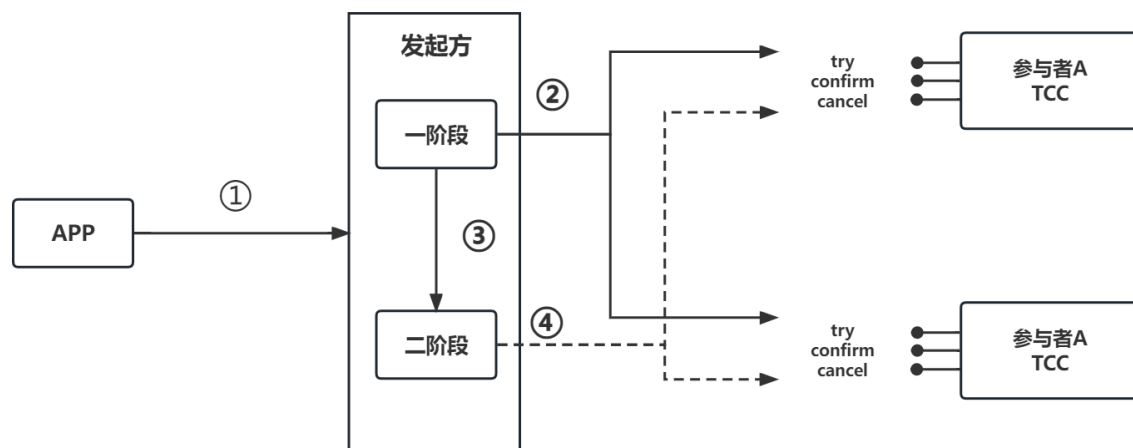
- 如果决议是全局提交，此时分支事务此时已经完成提交，不需要同步协调处理（只需要异步清理回滚日志），Phase2 可以非常快速地完成。
- 如果决议是全局回滚，RM 收到协调器发来的回滚请求，通过 XID 和 Branch ID 找到相应的回滚日志记录，通过回滚记录生成反向的更新 SQL 并执行，以完成分支的回滚。

AT模式的一阶段、二阶段提交和回滚均由Seata框架自动生成，用户只需编写“业务SQL”，便能轻松接入分布式事务，AT模式是一种对业务无任何侵入的分布式事务解决方案。

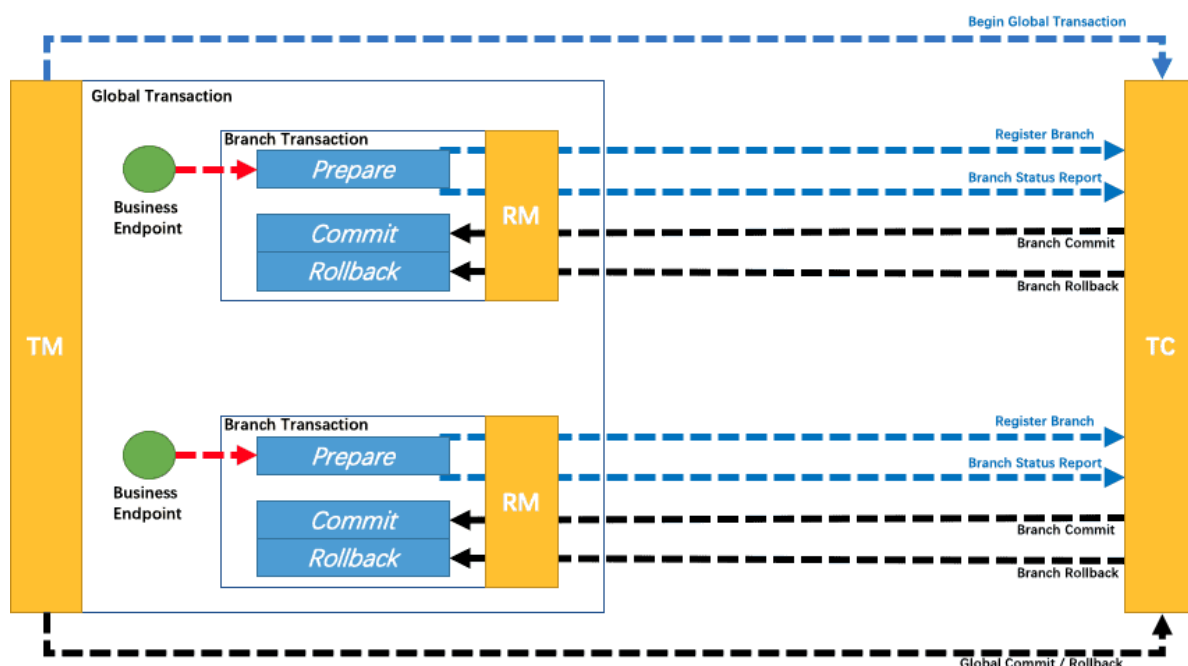
7.2.3 TCC模式

Seata中的TCC 模式包含三个阶段。

- Try 阶段：所有参与分布式事务的分支，对业务资源进行检查和预留。
- Confirm阶段：所有分支的Try全部成功后，执行业务提交。
- Cancel阶段：取消Try阶段预留的业务资源。



对比AT或者XA模式来说，TCC模式需要我们自己抽象并实现Try，Confirm，Cancel三个接口，编码量会大一些，但是由于事务的每一个阶段都由开发人员自行实现。而且相较于AT模式来说，减少了SQL解析的过程，也没有全局锁的限制，所以TCC模式的性能是优于AT、XA模式，但是带来的是工作量的增加。



根据两阶段行为模式的不同，我们将分支事务划分为 **Automatic (Branch) Transaction Mode** 和 **Manual (Branch) Transaction Mode**。

AT 模式 ([参考链接 TBD](#)) 基于 **支持本地 ACID 事务** 的 **关系型数据库**：

- 一阶段 prepare 行为：在本地事务中，一并提交业务数据更新和相应回滚日志记录。
- 二阶段 commit 行为：马上成功结束，**自动** 异步批量清理回滚日志。
- 二阶段 rollback 行为：通过回滚日志，**自动** 生成补偿操作，完成数据回滚。

相应的，TCC 模式，不依赖于底层数据资源的事务支持：

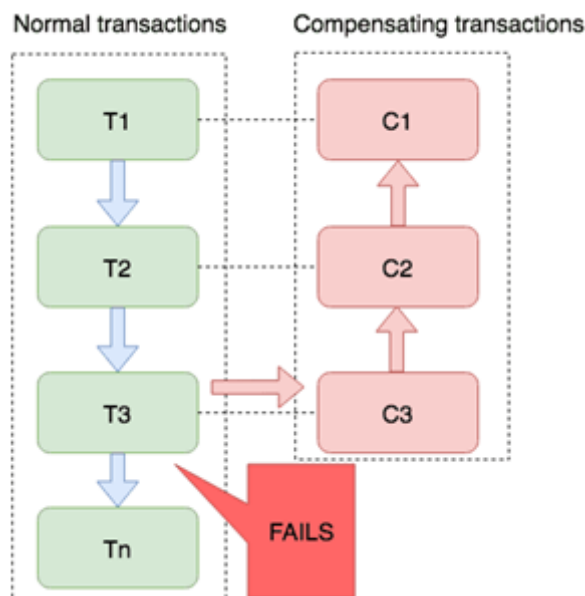
- 一阶段 prepare 行为：调用 **自定义** 的 prepare 逻辑。
- 二阶段 commit 行为：调用 **自定义** 的 commit 逻辑。

- 二阶段 rollback 行为：调用 **自定义** 的 rollback 逻辑。

所谓 TCC 模式，是指支持把 **自定义** 的分支事务纳入到全局事务的管理中。

7.2.4 Saga模式

Saga模式是1987年由普林斯顿大学的Hector Garcia-Molina和Kenneth Salem共同提出的。该模式不同于前面的模式，它不是2PC的，其是一种长事务解决方案。



其应用场景是：在无法提供TC、TM、RM接口的情况下，对于一个流程很长的复杂业务，其会包含很多的子流程（事务）。每个子流程都让它们真实提交它们真正的执行结果。只有当前子流程执行成功后才能执行下一个子流程。若整个流程中所有子流程全部执行成功，则整个业务流程成功；只要有一个子流程执行失败，则可采用两种补偿方式：

- 向后恢复：对于其前面所有成功的子流程，其执行结果全部撤销
- 向前恢复：重试失败的子流程直到其成功

7.2.1 XA模式

XA 模式是 Seata 另一种无侵入的分布式事务解决方案，XA模式在 Seata 定义的分布式事务框架内，利用事务资源（数据库、消息服务等）对 XA 协议的支持，以 XA 协议的机制来管理分支事务的一种事务模式，XA要求数据库本身提供对规范和协议的支持。

从编程模型上，XA 模式与 AT 模式保持完全一致。只需要修改数据源代理，即可实现 XA 模式与 AT 模式之间的切换。

代码如下所示：

```

1 @Bean("dataSource")
2 public DataSource dataSource(DruidDataSource druidDataSource) {
3     // DataSourceProxy for AT mode
4     // return new DataSourceProxy(druidDataSource);
5
6     // DataSourceProxyXA for XA mode
7     return new DataSourceProxyXA(druidDataSource);
8 }

```

7.3 Seata-Server的配置与启动

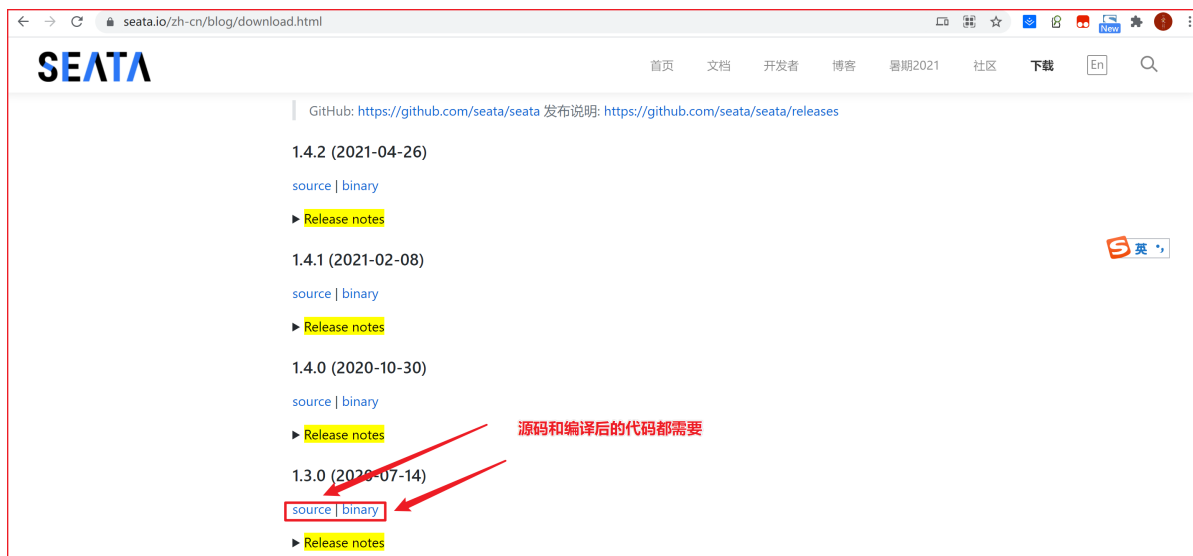
我们下面就以Seata默认的AT事务模式来实现分布式事务。

7.3.1 Seata下载

<https://seata.io/zh-cn/blog/download.html>

从官网下载Seata Server，源码与打过包的都需要下载。源码中包含很多需要运行的脚本文件，而打过包的则是可运行的服务器本身。

注意下载版本为：1.3.0



7.3.2 运行mysql.sql脚本

在seata源码解压目录的script/server/db下找到mysql.sql文件运行。该脚本会创建了三张表。这三张表都是用于保存整个系统中分布式事务相关日志数据的。



7.3.3 修改file.conf

修改seata-server解压目录下conf中的file.conf文件。该配置文件用于指定seata server 存放日志的位置。

```
1 ## transaction log store, only used in seata-server
2 store {
3     ## store mode: file, db, redis
4     mode = "db" 模式改为数据库模式
5     ## rsa decryption public key
6     publicKey = ""
7     ## file store property
8     file {
9         ## store location dir
10        dir = "sessionStore"
11        # branch session size , if exceeded first try compress lockkey, still exceeded throws exceptions
12        maxBranchSessionSize = 16384
13        # globe session size , if exceeded throws exceptions
14        maxGlobalSessionSize = 512
15        # file buffer size , if exceeded allocate new buffer
16        fileWriteBufferCacheSize = 16384
17        # when recover batch read size
18        sessionReloadReadSize = 100
19        # async, sync
20        flushDiskMode = async
21    }
22
23    ## database store property
24    db {
25        ## the implement of javax.sql.DataSource, such as DruidDataSource(druid)/BasicDataSource(dbcp)/HikariDataSource(hikari) etc.
26        datasource = "druid"
27        ## mysql/oracle/postgresql/h2/oceanbase etc.
28        dbType = "mysql"
29        driverClassName = "com.mysql.jdbc.Driver"
30        ## if using mysql to store the data, recommend add rewriteBatchedStatements=true in jdbc connection param
31        url = "jdbc:mysql://127.0.0.1:3306/seata?rewriteBatchedStatements=true"
32        user = "root"
33        password = "root" 数据库连接配置信息
34        minConn = 5
35        maxConn = 100
36        globalTable = "global_table"
37        branchTable = "branch_table"
38        lockTable = "lock_table"
39    }
40 }
```

7.3.4 修改registry.conf

修改seata-server解压目录下conf中的registry.conf文件。该配置文件用于指定seata要连接的注册中心与配置中心。

```
1 registry {
2     # file 、nacos 、eureka、redis、zk、consul、etcd3、sofa
3     type = "nacos"
4
5     nacos {
6         application = "seata-server"
7         serverAddr = "127.0.0.1:8848"
8         group = "SEATA_GROUP"
9         namespace = ""
10        cluster = "default"
11        username = "nacos"
12        password = "nacos"
13    }
14
15    56
16    config {
17        # file、nacos 、apollo、zk、consul、etcd3
18        type = "nacos"
19
20        nacos {
21            serverAddr = "127.0.0.1:8848"
22            namespace = ""
23            group = "SEATA_GROUP"
24            username = "nacos"
25            password = "nacos"
26            dataId = "seataServer.properties"
27        }
28    }
29 }
```

7.3.5 修改config.txt

将seata源码解压目录的script/config-center下的config.txt文件复制到seata-server解压 目录的根目录中，然后再进行修改

```

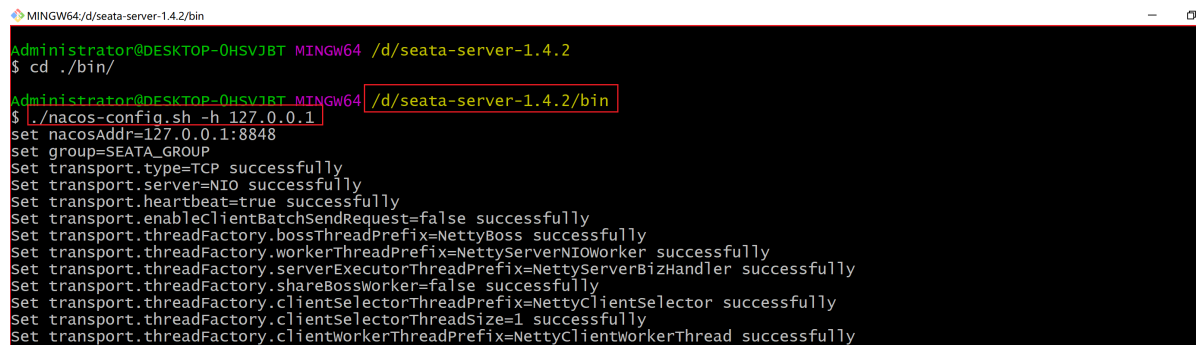
31 client.tm.defaultGlobalTransactionTimeout=60000
32 client.tm.degradeCheck=false
33 client.tm.degradeCheckAllowTimes=10
34 client.tm.degradeCheckPeriod=2000
35 store.mode=db
36 store.publicKey=
37 store.file.dir=file_store/data
38 store.file.maxBranchSessionSize=16384
39 store.file.maxGlobalSessionSize=512
40 store.file.fileWriteBufferCacheSize=16384
41 store.file.flushDiskMode=async
42 store.file.sessionReloadReadSize=100
43 store.db.datasource=druid
44 store.db.dbType=mysql
45 store.db.driverClassName=com.mysql.jdbc.Driver
46 store.db.url=jdbc:mysql://127.0.0.1:3306/seata?useUnicode=true&rewriteBatchedStatements=true
47 store.db.user=root
48 store.db.password=root
49 store.db.minConn=5

```

7.3.6 运行nacos-config.sh脚本

在seata源码解压目录的script/config-center/nacos下有nacos-config.sh脚本文件。将该脚本文件复制到config.txt的下一级目录中。

```
1 sh nacos-config.sh -h 127.0.0.1
```



```

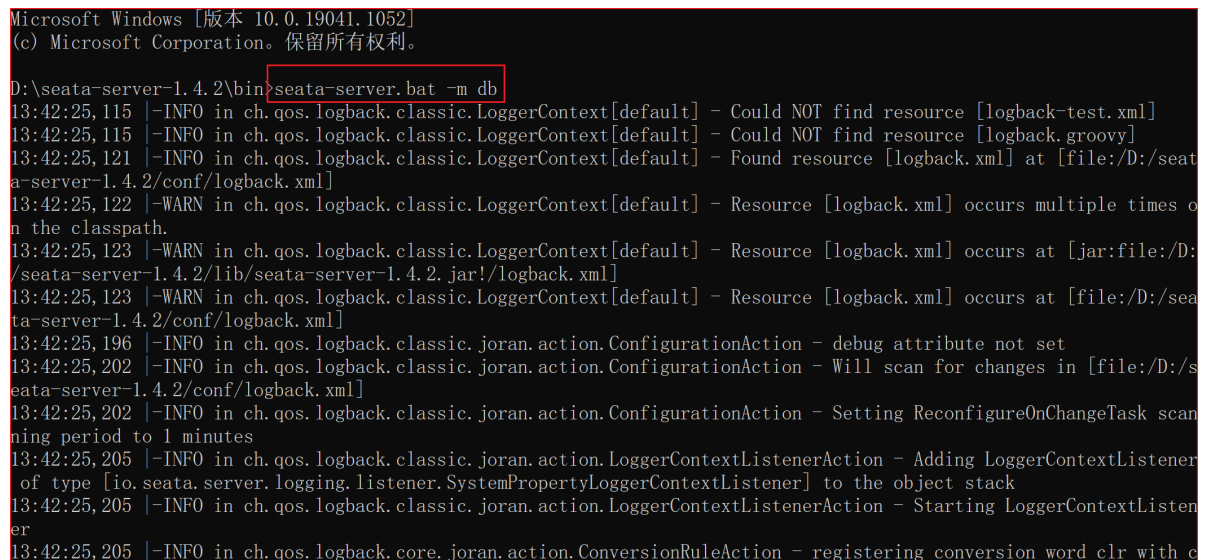
MINGW64/d/seata-server-1.4.2/bin
Administrator@DESKTOP-0HSVJ8T MINGW64 /d/seata-server-1.4.2
$ cd ./bin/
Administrator@DESKTOP-0HSVJ8T MINGW64 /d/seata-server-1.4.2/bin
$ ./nacos-config.sh -h 127.0.0.1
set nacosAddr=127.0.0.1:8848
set group=SEATA_GROUP
set transport.type=TCP successfully
set transport.server=NIO successfully
set transport.heartbeat=true successfully
set transport.enableClientBatchSendRequest=false successfully
set transport.threadFactory.bossThreadPrefix=NettyBoss successfully
set transport.threadFactory.workerThreadPrefix=NettyServerNIOworker successfully
set transport.threadFactory.serverExecutorThreadPrefix=NettyServerBizHandler successfully
set transport.threadFactory.shareBossWorker=false successfully
set transport.threadFactory.clientSelectorThreadPrefix=NettyClientSelector successfully
set transport.threadFactory.clientSelectorThreadSize=1 successfully
set transport.threadFactory.clientWorkerThreadPrefix=NettyClientWorkerThread successfully

```

7.3.7 启动Seata-server

在seata-server解压目录下的bin目录中有个文件seata-server.bat，在命令行运行这个批处理文件。

```
1 seata-server.bat -m db -p 8091 -h 127.0.0.1
```



```

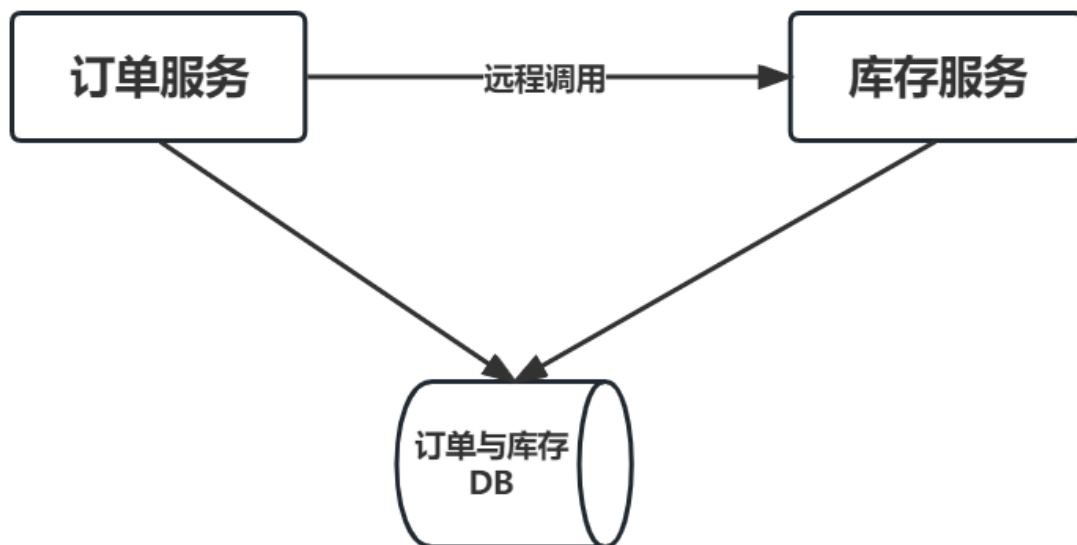
Microsoft Windows [版本 10.0.19041.1052]
(c) Microsoft Corporation. 保留所有权利。

D:\seata-server-1.4.2\bin>seata-server.bat -m db
13:42:25,115 |-INFO in ch.qos.logback.classic.LoggerContext[default] - Could NOT find resource [logback-test.xml]
13:42:25,115 |-INFO in ch.qos.logback.classic.LoggerContext[default] - Could NOT find resource [logback.groovy]
13:42:25,121 |-INFO in ch.qos.logback.classic.LoggerContext[default] - Found resource [logback.xml] at [file:D:/seata-server-1.4.2/conf/logback.xml]
13:42:25,122 |-WARN in ch.qos.logback.classic.LoggerContext[default] - Resource [logback.xml] occurs multiple times on the classpath.
13:42:25,123 |-WARN in ch.qos.logback.classic.LoggerContext[default] - Resource [logback.xml] occurs at [jar:file:D:/seata-server-1.4.2/lib/seata-server-1.4.2.jar!/logback.xml]
13:42:25,123 |-WARN in ch.qos.logback.classic.LoggerContext[default] - Resource [logback.xml] occurs at [file:D:/seata-server-1.4.2/conf/logback.xml]
13:42:25,196 |-INFO in ch.qos.logback.classic.joran.action.ConfigurationAction - debug attribute not set
13:42:25,202 |-INFO in ch.qos.logback.classic.joran.action.ConfigurationAction - Will scan for changes in [file:D:/seata-server-1.4.2/conf/logback.xml]
13:42:25,202 |-INFO in ch.qos.logback.classic.joran.action.ConfigurationAction - Setting ReconfigureOnChangeTask scanning period to 1 minutes
13:42:25,205 |-INFO in ch.qos.logback.classic.joran.action.LoggerContextListenerAction - Adding LoggerContextListener of type [io.seata.server.logging.listener.SystemPropertyLoggerContextListener] to the object stack
13:42:25,205 |-INFO in ch.qos.logback.classic.joran.action.LoggerContextListenerAction - Starting LoggerContextListener
13:42:25,205 |-INFO in ch.qos.logback.core.joran.action.ConversionRuleAction - registering conversion word clr with c

```


7.4 测试环境搭建

7.4.1 需求



- 订单服务-->采购服务
- 库存服务-->库存服务

使用DRP系统中的purchase采购模块调用stock库存模块来模拟消费者调用提供者。

当采购了新的生产资料时，会调用采购模块，增加采购表purchase中的相应生产资料数量，并调用库存模块增加库存表stock中相应生产资料数量。只有这样才能保证了数据的一致。

7.4.2 库存工程07-drp-stock

该工程充当着Provider工程。

(1) 定义工程

复制03-payment-nacos工程，并重命名为07-drp-stock。将原来工程中的所有类删除，保留原有的配置文件。

(2) 定义Controller、Service和Dao

定义StockController类

```
1  @RestController
2  public class StockController {
3      @Autowired
4      private StockService stockService;
5
6      @RequestMapping("/stock/add")
7      public boolean addStockHandle(@RequestBody Stock stock) {
8          return stockService.addStock(stock);
9      }
10 }
```

定义StockService接口 和 实现类


```

1  public interface StockService {
2      //新增库存
3      boolean addStock(Stock stock);
4      //根据名称查询库存
5      Stock getStockByName(String name);
6  }
7  @Service
8  public class StockServiceImpl implements StockService {
9
10     @Autowired
11     StockRepository stockRepository;
12     @Override
13     public boolean addStock(Stock stock) {
14         Stock s = stockRepository.findByName(stock.getName());
15         if (s == null) {
16             stockRepository.save(stock);
17             return true;
18         } else {
19             s.setTotal(s.getTotal() + stock.getTotal());
20             stockRepository.save(s);
21         }
22         return true;
23     }
24
25     @Override
26     public Stock getStockByName(String name) {
27         return stockRepository.findByName(name);
28     }
29 }

```

实体类Stock

```

1  @Data
2  @NoArgsConstructor
3  @AllArgsConstructor
4  @Entity
5  @JsonIgnoreProperties({"hibernateLazyInitializer","handler","fieldHandler"})
6  public class Stock {
7
8      @Id
9      @GeneratedValue(strategy = GenerationType.IDENTITY)
10     private Integer id;
11     private String name; //库存名称
12     private Integer total; //资源数量
13 }

```

定义StockRepository接口

```

1  public interface StockRepository extends JpaRepository<Stock,Integer> {
2      //根据名称查询库存
3      Stock findByName(String name);
4  }

```

(3) 定义启动类

```
1 @SpringBootApplication
2 public class SeataStockApplication {
3
4     public static void main(String[] args) {
5         SpringApplication.run(SeataStockApplication.class, args);
6     }
7 }
```

(4) 修改配置文件

```
1 # 配置配置中心的服务地址
2 spring:
3     application:
4         # 配置应用名称
5         name: msc-stock
6     cloud:
7         nacos:
8             config:
9                 server-addr: localhost:8848
10                file-extension: yaml
11                group: SEATA_GROUP
```

(5) 在nacos-config中新建配置文件

public				
配置管理 public 查询结果: 共查询到 2 条满足要求的配置。				
Data ID: *.yaml		Group: SEATA_GROUP	查询	高级查询 ▾ 导入配置 +
☐	Data Id ⚡	Group ⚡	归属应用: ⚡	操作
☐	msc-stock.yaml	SEATA_GROUP		详情 示例代码 编辑 删除 更多
☐	msc-purchase.yaml	SEATA_GROUP		详情 示例代码 编辑 删除 更多

该文件的具体内容如下

```
1 server:
2     port: 8081
3     spring:
4         cloud:
5             nacos:
6                 discovery:
7                     server-addr: 127.0.0.1:8848
8     jpa:
9         generate-ddl: true
10        show-sql: true
11        hibernate:
12            ddl-auto: none
13    datasource:
14        type: com.alibaba.druid.pool.DruidDataSource
15        driver-class-name: com.mysql.jdbc.Driver
```

```

16     url: jdbc:mysql://127.0.0.1:3306/test?
    useUnicode=true&characterEncoding=utf8
17     username: root
18     password: root
19
20 logging:
21     level:
22         root: info
23         org.hibernate: info
24         org.hibernate.type.descriptor.sql.BasicBinder: trace
25         org.hibernate.type.descriptor.sql.BasicExtractor: trace
26         com.abc.provider: debug
27

```

7.4.3 定义采购工程07-drp-purchase

该工程充当着Consumer工程。

(1) 定义工程

复制07-drp-stock工程，并重命名为07-drp-purchase。将原来工程中的所有类删除，保留原有的配置文件

(2) 定义Controller、Service和Dao

定义PurchaseController类

```

1  @RestController
2  public class PurchaseController {
3
4      @Autowired
5      private PurchaseService purchaseService;
6      @Autowired
7      private RestTemplate restTemplate;
8
9      @RequestMapping("/purchase/add/{deno}")
10     public boolean addPurchaseHandle(@RequestBody Purchase purchase
11         ,@PathVariable("deno") int deno) {
12         purchaseService.addPurchase(purchase);
13
14         Stock stock = new Stock();
15         stock.setName(purchase.getName());
16         stock.setTotal(purchase.getCount());
17
18         String url = "http://msc-stock/stock/add";
19         Boolean result = restTemplate.postForObject(url, stock,
20             Boolean.class);
21         //手动异常
22         int i = 3 / deno;
23         if (result != null) {
24             return result;
25         }
26         return false;
27     }
28 }

```

```
27  
28 }
```

定义PurchaseService接口和实现类

```
1  public interface PurchaseService {  
2      //新增库存  
3      boolean addPurchase(Purchase purchase);  
4      //根据名称查询库存  
5      Purchase getPurchaseByName(String name);  
6  }  
7  @Service  
8  public class PurchaseServiceImpl implements PurchaseService {  
9      @Autowired  
10     PurchaseRepository purchaseRepository;  
11  
12     @Override  
13     public boolean addPurchase(Purchase purchase) {  
14         Purchase p = purchaseRepository.findByName(purchase.getName());  
15         if (p == null) {  
16             purchaseRepository.save(purchase);  
17             return true;  
18         } else {  
19             p.setCount(p.getCount() + purchase.getCount());  
20             purchaseRepository.save(p);  
21         }  
22         return true;  
23     }  
24  
25     @Override  
26     public Purchase getPurchaseByName(String name) {  
27         return purchaseRepository.findByName(name);  
28     }  
29 }
```

定义PurchaseRepository接口

```
1  public interface PurchaseRepository extends JpaRepository<Purchase, Integer> {  
2      //根据名称查询库存  
3      Purchase findByName(String name);  
4  }
```

定义实体类Purchase

```

1  @Data
2  @NoArgsConstructor
3  @AllArgsConstructor
4  @Entity
5  @JsonIgnoreProperties({"hibernateLazyInitializer","handler","fieldHandler"})
6  public class Purchase {
7      @Id
8      @GeneratedValue(strategy = GenerationType.IDENTITY)
9      private Integer id;
10     private String name;//采购资源名称
11     private Integer count;//采购量
12 }

```

定义实体类Stock

```

1  @Data
2  @Builder
3  public class Stock {
4
5      private Integer id;
6      private String name;//库存名称
7      private Integer total;//资源数量
8  }
9

```

(3) 定义启动类

```

1  @SpringBootApplication
2  public class SeataPurchaseApplication {
3      public static void main(String[] args) {
4          SpringApplication.run(SeataPurchaseApplication.class, args);
5      }
6      @Bean
7      @LoadBalanced
8      public RestTemplate restTemplate() {
9          return new RestTemplate();
10     }
11 }

```

(4) 修改配置文件

```

1  spring:
2      application:
3          name: msc-purchase
4      cloud:
5          nacos:
6              config:
7                  server-addr: localhost:8848
8                  file-extension: yml
9                  group: SEATA_GROUP

```

(5) 在nacos-config中新建配置文件

配置管理 | public 查询结果: 共查询到 2 条满足要求的配置。

Data ID: *.yml Group: SEATA_GROUP 查询 高级查询 导入配置 +

☐	Data Id ↓↑	Group ↓↑	归属应用: ↓↑	操作
☐	msc-stock.yml	SEATA_GROUP		详情 示例代码 编辑 删除 更多
☐	msc-purchase.yml	SEATA_GROUP		详情 示例代码 编辑 删除 更多

该文件的具体内容如下

```
1  spring:
2    cloud:
3      nacos:
4        discovery:
5          server-addr: 127.0.0.1:8848
6    jpa:
7      generate-ddl: true
8      show-sql: true
9      hibernate:
10        ddl-auto: none
11    datasource:
12      type: com.alibaba.druid.pool.DruidDataSource
13      driver-class-name: com.mysql.jdbc.Driver
14      url: jdbc:mysql://127.0.0.1:3306/test?
15        useUnicode=true&characterEncoding=utf8
16      username: root
17      password: root
18    logging:
19      level:
20        root: info
21        org.hibernate: info
22        org.hibernate.type.descriptor.sql.BasicBinder: trace
23        org.hibernate.type.descriptor.sql.BasicExtractor: trace
24        com.abc.provider: debug
```

7.5 配置Seata-Client

Seata-Client, 即安装有Seata依赖的应用。

7.5.1 创建两个工程

为了保留下原始代码, 我们这里复制07-drp-stock与07-drp-purchase两个工 程, 分别重新命名为07-seata-stock与07-seata-purchase, 将这两个新工程改造为 Seata-Client。

7.5.2 添加undo_log表

在客户端处理的业务相关的每个数据库中都要添加undo_log表，用于保存需要回滚的数据。建表语句脚本在seata源码解压目录的script/client/at/db目录下的mysql.sql中。

7.5.3 添加依赖

在pom中添加seata的依赖。

```
1 <!-- seata依赖 -->
2 <dependency>
3     <groupId>io.seata</groupId>
4     <artifactId>seata-spring-boot-starter</artifactId>
5     <version>1.3.0</version>
6 </dependency>
7 <dependency>
8     <groupId>com.alibaba.cloud</groupId>
9     <artifactId>spring-cloud-starter-alibaba-seata</artifactId>
10    <version>2.2.1.RELEASE</version>
11    <exclusions>
12        <exclusion>
13            <groupId>io.seata</groupId>
14            <artifactId>seata-spring-boot-starter</artifactId>
15        </exclusion>
16    </exclusions>
17 </dependency>
```

7.5.4 修改bootstrap.yml

在每个seata client工程的bootstrap.yml中指定要使用的事务服务组。

```
1 spring:
2   application:
3     name: msc-purchase
4   cloud:
5     nacos:
6       config:
7         server-addr: localhost:8848
8         file-extension: yaml
9         group: SEATA_GROUP
10  seata:
11    tx-service-group: my_test_tx_group
12
13    logging.level.io.seata: debug
14
```

```
1 seata:
2   tx-service-group: my_test_tx_group
3
4 logging.level.io.seata: debug
```

7.5.5 数据源代理

```
1 @Configuration
2 public class DataSourceConfig {
3     @Bean
4     @ConfigurationProperties(prefix = "spring.datasource")
5     public DruidDataSource druidDataSource() {
6         return new DruidDataSource();
7     }
8 }
```

```

7      }
8
9      /**
10     * 需要将 DataSourceProxy 设置为主数据源，否则事务无法回滚
11     * @param druidDataSource The DruidDataSource
12     * @return The default datasource
13     */
14     @Primary
15     @Bean("dataSource")
16     public DataSource dataSource(DruidDataSource druidDataSource) {
17         return new DataSourceProxy(druidDataSource);
18     }
19 }

```

7.5.6 添加@GlobalTransactional注解

在需要开启分布式事务的方法上添加@GlobalTransactional注解。本例就是在07-seata-purchase工程的PurchaseController的addPurchaseHandle()方

法上添加@GlobalTransactional注解。

```

25
26     @GlobalTransactional
27     @RequestMapping("/purchase/add/{deno}")
28     public boolean addPurchaseHandle(@RequestBody Purchase purchase
29                                     ,@PathVariable("deno") int deno) {
30         purchaseService.addPurchase(purchase);
31
32         Stock stock = new Stock();
33         stock.setName(purchase.getName());
34         stock.setTotal(purchase.getCount());
35
36         String url = "http://msc-stock/stock/add";
37         Boolean result = restTemplate.postForObject(url, stock, Boolean.class);
38         // 手动异常
39         int i = 3 / deno;
40         if (result != null) {
41             return result;
42         }
43         return false;
44     }
45

```

配置分支事务的注解@Transactional